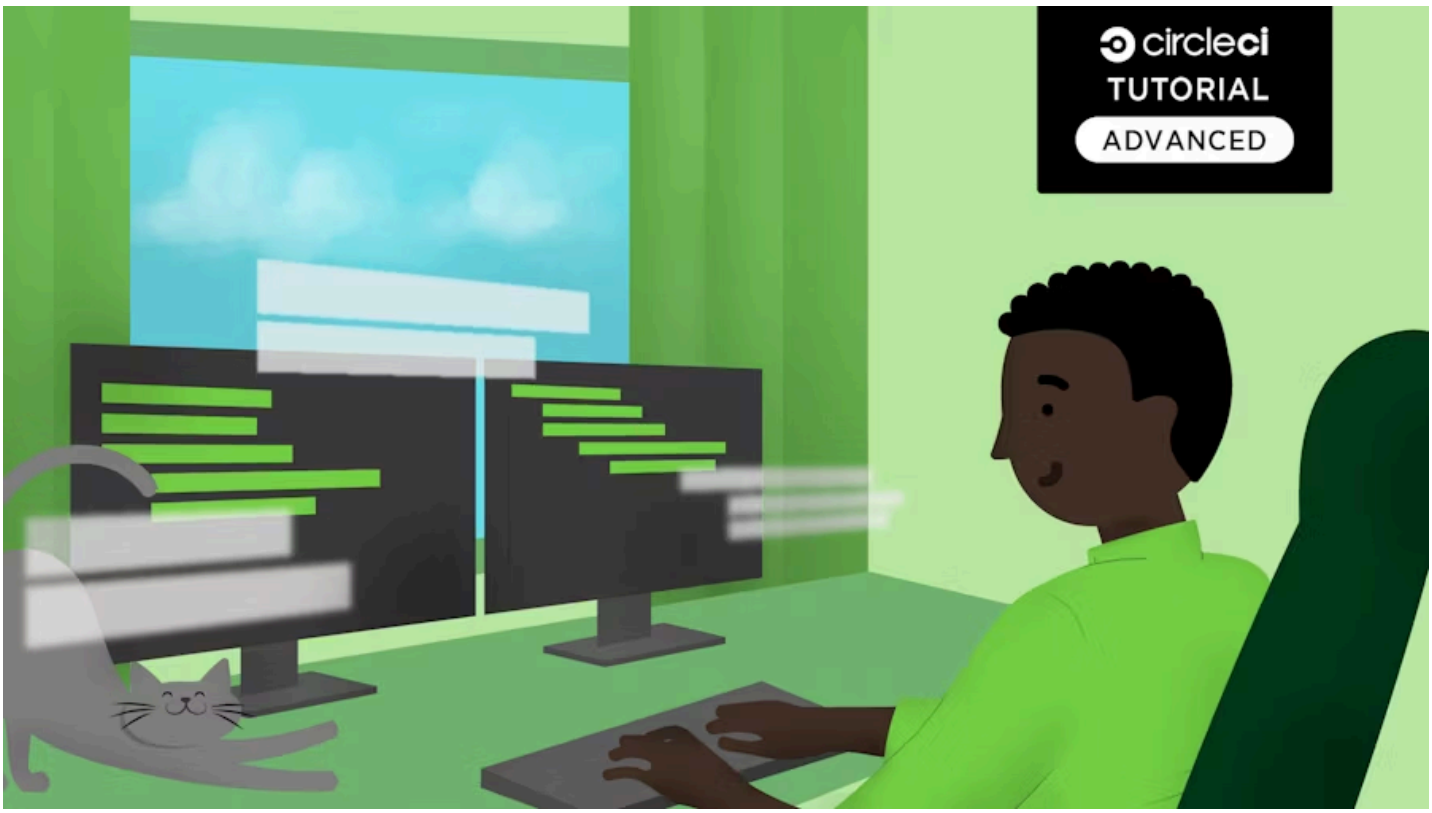
TUTORIALS | PUBLISHED OCT 25, 2020 | 8 MIN READ

Building Docker images for multiple operating system architectures

Angel Rivera (/blog/author/angel-rivera/)
Developer Advocate, CircleCI



There are often circumstances where software is compiled and packaged into artifacts that must function on multiple operating systems (OS) (https://en.wikipedia.org/wiki/Operating_system) and processor architectures (https://en.wikipedia.org/wiki/Microarchitecture). It is almost impossible to execute an application on a different OS/architecture platform than the one it was designed for. That's why it's a common practice to build releases for many different platforms. This can be difficult to accomplish when the platform you are using to build artifacts is different from the platforms you want to target for deployment. For instance, developing an application on Windows and deploying it to Linux and macOS machines involves provisioning and configuring build machines for each of the operating systems and architecture platforms you're targeting. Multi-OS builds within pipelines can be achieved using a variety of techniques, but due to the stringent characteristics of processor architectures, artifacts must be compiled and produced on the same hardware that they are targeting.

Docker (https://www.docker.com/) is a modern way to package applications into immutable and deployable artifacts in the form of Docker images (https://docs.docker.com/engine/reference/commandline/images/) and containers (https://www.docker.com/resources/what-container). As with traditional artifact packaging, Docker images also experience the same processor architecture build constraints. Docker images must also be built on the hardware architectures they're intended to run on. In this post I'll discuss how to build Docker images within CI pipelines (https://circleci.com/blog/what-is-a-ci-cd-pipeline/) that target multiple processor architectures such as linux/amd64, linux/arm64, linux/riscv64, etc.

Getting started

Let's take a look at an example code repository (https://github.com/metcalfc/docker-circleci-example), built by Chad Metcalf, that demonstrates how to package an application into multi-architecture Docker images. We're only going to focus on the continuous integration (https://circleci.com/continuous-integration/) aspects of building these multi-architecture Docker images. The CircleCI `config.yml` file defines the CI pipeline build instructions. It is found in the `.circleci/config.yml` directory. In this post I'm going to focus on the `.circleci/config.yml` file and the `Makefile` file from this repo.

Makefiles (https://www.gnu.org/prep/standards/html_node/Makefile-Basics.html#Makefile-Basics) can be viewed as build/compile directives that are required by the make utility (https://www.gnu.org/software/make/) that automates the build processes. The `Makefile` in this project contains the directives and commands that are executed from the CI pipeline.

Using Docker Buildx

Before I go deeper into the `Makefile` and `config.yml` file break downs, I want to take a moment to discuss Buildx (https://docs.docker.com/buildx/working-with-buildx/), which is a currently a CLI plugin that extends the Docker CLI with the full set of features provided by the Moby BuildKit builder toolkit. It provides the same user experience as `docker build` with many new features like creating scoped builder instances and building against multiple nodes concurrently.

At the time of this writing, the Buildx feature is still in the **experimental** status, and requires a few environment configurations on the machine where Docker images will be built. The following are Buildx install directions for Docker version `19.03` and higher. The complete Buildx installation instructions can be found here (https://github.com/docker/buildx#installing), and below are the TL;DR instructions for a Linux machine with Docker 19.03 installed. The following commands compile and build the `Buildx` binary from source and installs it into the Docker plugin directory:

```
export DOCKER_BUILDKIT=1
docker build --platform=local -o . git://github.com/docker/buildx
mkdir -p ~/.docker/cli-plugins
mv buildx ~/.docker/cli-plugins/docker-buildx
```

You can also download the latest Buildx binaries for your OS here (https://github.com/docker/buildx/releases), and install it using these Buildx release binary directions (https://github.com/docker/buildx#binary-release).

After installing Buildx on your Docker builder machine, you can take advantage of all the Buildx capabilities:

- Multiple builder instance support (https://github.com/docker/buildx#working-with-builder-instances)
- Multi-node builds for cross-platform images (https://github.com/docker/buildx#building-multi-platform-images)
- Compose build support (https://github.com/docker/buildx#high-level-build-options)

I suggest you take the time to get better familiar with Buildx features. It is an essential technology when building multi-architecture Docker images, and it is heavily used in the examples below.

Configuring your CI pipeline

The `config.yml` file in the example project leverages the `Makefile` file and its functionality to execute the appropriate commands to complete the multi-architecture builds. This `config.yml` demonstrates how to leverage a single build job using a machine executor (https://circleci.com/docs/executor-types/#using-machine). This may seem a bit out of the norm since CircleCI provides the ability to build Docker images using the Docker executor (https://circleci.com/docs/executor-types/#using-docker).

The Docker platform leverages sharing and managing its host operating system kernels (https://docs.docker.com/get-started/overview/) vs. the kernel emulation found in virtual machines (VMs) (https://en.wikipedia.org/wiki/Kernel-based_Virtual_Machine). Since running Docker containers share the host OS kernel, they are architecturally very different from VMs. VMs are not based on container technology. They are made up of the user-spaces and kernel-spaces of an operating system. VM server hardware is virtualized, and each VM has its own isolated OS and apps. It shares hardware resources from the host, and can emulate various processor architectures/kernels within the VM. The kernel and hardware emulation capabilities of VMs are the main reasons the `machine executor` is the best choice for building multi-architecture Docker images.

Let's take a look at the `config.yml` in the example project:

```
version: 2.1
jobs:
  build:
    machine:
      image: ubuntu-1604:202007-01
    environment:
      DOCKER_BUILDKIT: 1
      BUILDX_PLATFORMS: linux/amd64,linux/arm64,linux/ppc64le,linux/s390x,linu
x/386,linux/arm/v7,linux/arm/v6
    steps:
      - checkout
      - run:
          name: Unit Tests
          command: make test
      - run:
          name: Log in to docker hub
          command: |
            docker login -u $DOCKER_USER -p $DOCKER_PASS
      - run:
          name: Build from dockerfile
          command: |
            TAG=edge make build
      - run:
          name: Push to docker hub
          command: |
            TAG=edge make push
      - run:
          name: Compose Up
          command: |
            TAG=edge make run
      - run:
          name: Check running containers
          command: |
            docker ps -a
      - run:
          name: Check logs
          command: |
            TAG=edge make logs
      - run:
          name: Compose down
          command: |
            TAG=edge make down
      - run:
          name: Install buildx
          command: |
```

```
            BUILDX_BINARY_URL="https://github.com/docker/buildx/releases/downlo
ad/v0.4.2/buildx-v0.4.2.linux-amd64"

            curl --output docker-buildx \
              --silent --show-error --location --fail --retry 3 \
              "$BUILDX_BINARY_URL"

            mkdir -p ~/.docker/cli-plugins

            mv docker-buildx ~/.docker/cli-plugins/
            chmod a+x ~/.docker/cli-plugins/docker-buildx

            docker buildx install
            # Run binfmt
            docker run --rm --privileged tonistiigi/binfmt:latest --install "$B
UILDX_PLATFORMS"
      - run:
          name: Tag golden
          command: |
            BUILDX_PLATFORMS="$BUILDX_PLATFORMS" make cross-build
```

As you may have noticed, most of the `command:` keys in this config file execute the functions defined in the `Makefile`. This pattern produces much less YAML syntax in the config file, but does complicate what's actually being executed in the `Makefile`.

Next, I'm going to focus on explaining the critical `command:` keys in this config file.

```
version: 2.1
jobs:
  build:
    machine:
      image: ubuntu-1604:202007-01
    environment:
      DOCKER_BUILDKIT: 1
      BUILDX_PLATFORMS: linux/amd64,linux/arm64,linux/ppc64le,linux/s390x,linu
x/386,linux/arm/v7,linux/arm/v6
    steps:
      - checkout
      - run:
          name: Unit Tests
          command: make test
      - run:
          name: Log in to docker hub
          command: |
            docker login -u $DOCKER_USER -p $DOCKER_PASS
      - run:
          name: Build from dockerfile
          command: |
            TAG=edge make build
      - run:
          name: Push to docker hub
          command: |
            TAG=edge make push
```

In the above code, the build is using a `machine executor` and assigning values to the `DOCKER_BUILDKIT` variable that enables Docker access to the experimental features and Buildx. The `BUILDX_PLATFORMS` variable is the list of OS and processor architectures that will produce Docker images. This list is targeting the Linux OS and a variety of processor architectures.

The remaining `run:` and `command:` keys demonstrate how to execute the application's unit tests, authenticate to Docker Hub in order to pull and push images, build a Docker image using the `Dockerfile` found in the `/app` directory, and push that image to Docker Hub.

NOTE: *The `docker login` step above ensures that your requests to Docker Hub are authenticated. Whenever you pull images from, or push images to, Docker Hub with CircleCI, we recommend **logging in to your Docker Hub account** for both `docker pull` and `docker push` steps in your CircleCI config. Logging in will make sure that your jobs have access to a higher Docker Hub rate limit (https://docs.docker.com/docker-hub/download-rate-limit/).*

```
      - run:
          name: Install buildx
          command: |
            BUILDX_BINARY_URL="https://github.com/docker/buildx/releases/downlo
ad/v0.4.2/buildx-v0.4.2.linux-amd64"

            curl --output docker-buildx \
              --silent --show-error --location --fail --retry 3 \
              "$BUILDX_BINARY_URL"

            mkdir -p ~/.docker/cli-plugins

            mv docker-buildx ~/.docker/cli-plugins/
            chmod a+x ~/.docker/cli-plugins/docker-buildx

            docker buildx install
            # Run binfmt
            docker run --rm --privileged tonistiigi/binfmt:latest --install "$B
UILDX_PLATFORMS"
```

In the code snippet above, the Buildx feature is being used to install the Buildx binary and configure it for usage in the executor. The Buildx tool can build multi-architecture images using a variety of strategies but the easiest method is to use Qemu emulation (https://wiki.qemu.org/Main_Page). It is a generic, open source machine emulator and virtualizer. When BuildKit needs to run a binary for a different architecture, it will automatically load it through a binary registered in the `binfmt_misc` handler. For QEMU binaries registered with `binfmt_misc` on the host OS to work transparently inside containers, they must be registed with the `fix_binary` flag.

The `docker run --rm --privileged tonistiigi/binfmt:latest --install "$BUILDX_PLATFORMS"` command pulls and spawns a binfmt (https://github.com/tonistiigi/binfmt#installing-emulators) container for every platform listed in the `$BUILD_PLATFORMS` variable defined earlier in the file.

```
      - run:
          name: Tag golden
          command: |
            BUILDX_PLATFORMS="$BUILDX_PLATFORMS" make cross-build
```

The above code snippet specifies the last command to execute in the pipeline. It builds the multi-architecture Docker images we want to target. The `command:` key is making a call to the `cross-build` function defined inside the `Makefile`, so let's take a look at the underlying commands associated with

this function.

```
# Makefile cross-build function

.PHONY: cross-build
cross-build:
	@docker buildx create --name mybuilder --use
	@docker buildx build --platform ${BUILDX_PLATFORMS} -t ${PROD_IMAGE} --
push ./app
```

The code snippet above is the actual `cross-build` `make` command, which creates a new Buildx builder instance. It follows that with the `docker buildx build` command that triggers the process to build an individual Docker image for every platform listed in the `${BUILDX_PLATFORMS}` environment variable. This is fed into the `--platform` flag of the command. The `-t` flag tags/names the Docker images and the `--push` flag will automatically push the build result to a Docker registry. In this case, it is Docker Hub.

Summary

This post demonstrated how to build various Docker images for multiple operating systems and processor architectures from within a CI pipeline. This post also briefly introduced the Docker Buildx feature (https://docs.docker.com/buildx/working-with-buildx/), which is currently an experimental utility that is expected to become the defacto build utility in future releases of Docker. I consider Buildx to be the next-gen Docker image building tool that will enable expansive, advanced, and optimized capabilities to enhance the current image building experience.

I also briefly discussed some of the intricacies of building Docker images that target multiple operating systems and platform architectures, which highlight the technical differences between Docker containers and VMs. Though seemingly similar at an abstract view, they are fundamentally different at their cores. Finally, I'll reiterate that **Docker has implemented new Docker Hub rate limits (https://docs.docker.com/docker-hub/download-rate-limit/)** which requires all calls to Docker Hub to be authenticated. Whenever you pull images from, or push images to, Docker Hub on CircleCI, we recommend **logging in to your Docker Hub account** for both `docker pull` and `docker push` steps in your CircleCI config.

Thank you for following this post and I hope you found it useful. Please feel free to reach out with feedback on Twitter @punkdata (https://twitter.com/punkdata).

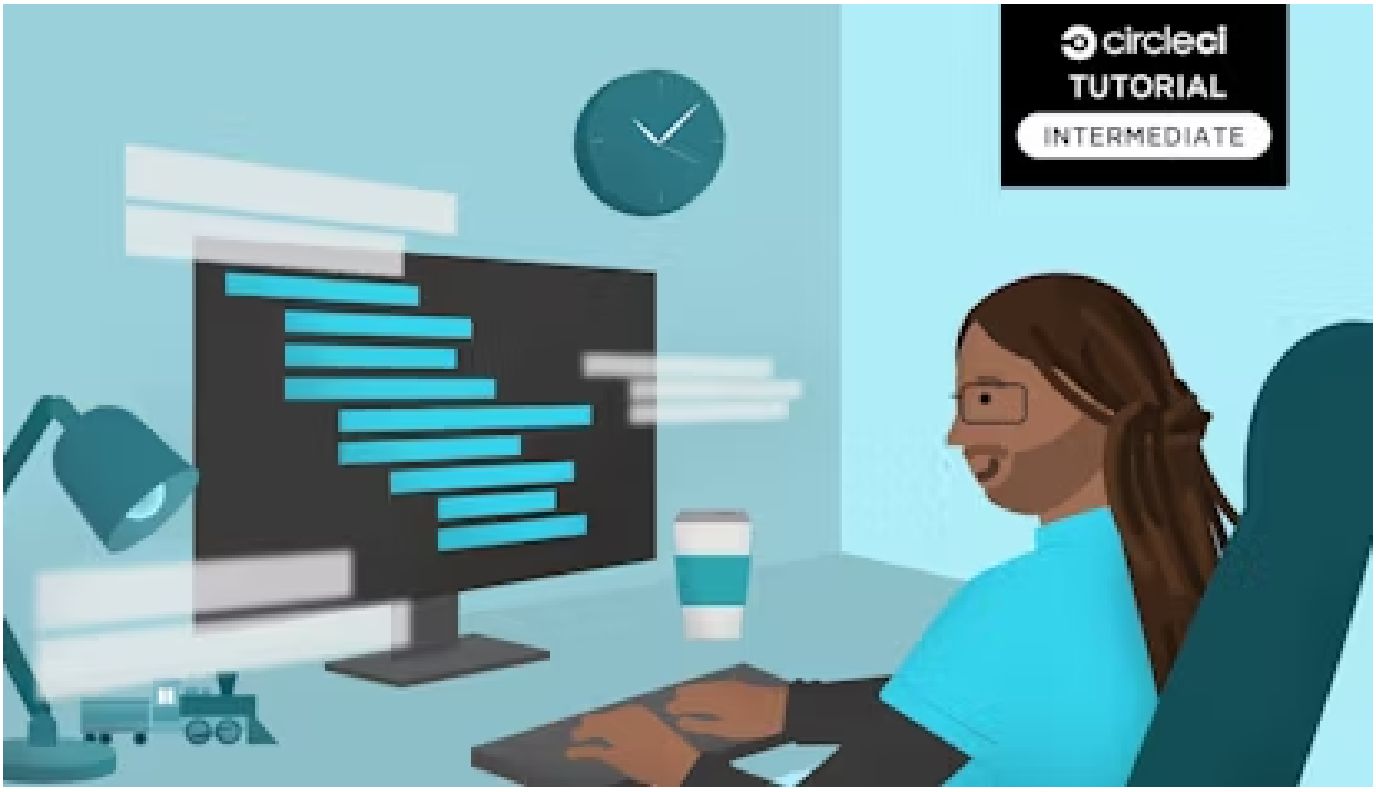
Tutorials (/blog/tag/tutorials/)

Engineering (/blog/tag/engineering/)

Security (/blog/tag/security/)

DevOps 101 (/blog/tag/devops101/)

Similar posts you may enjoy



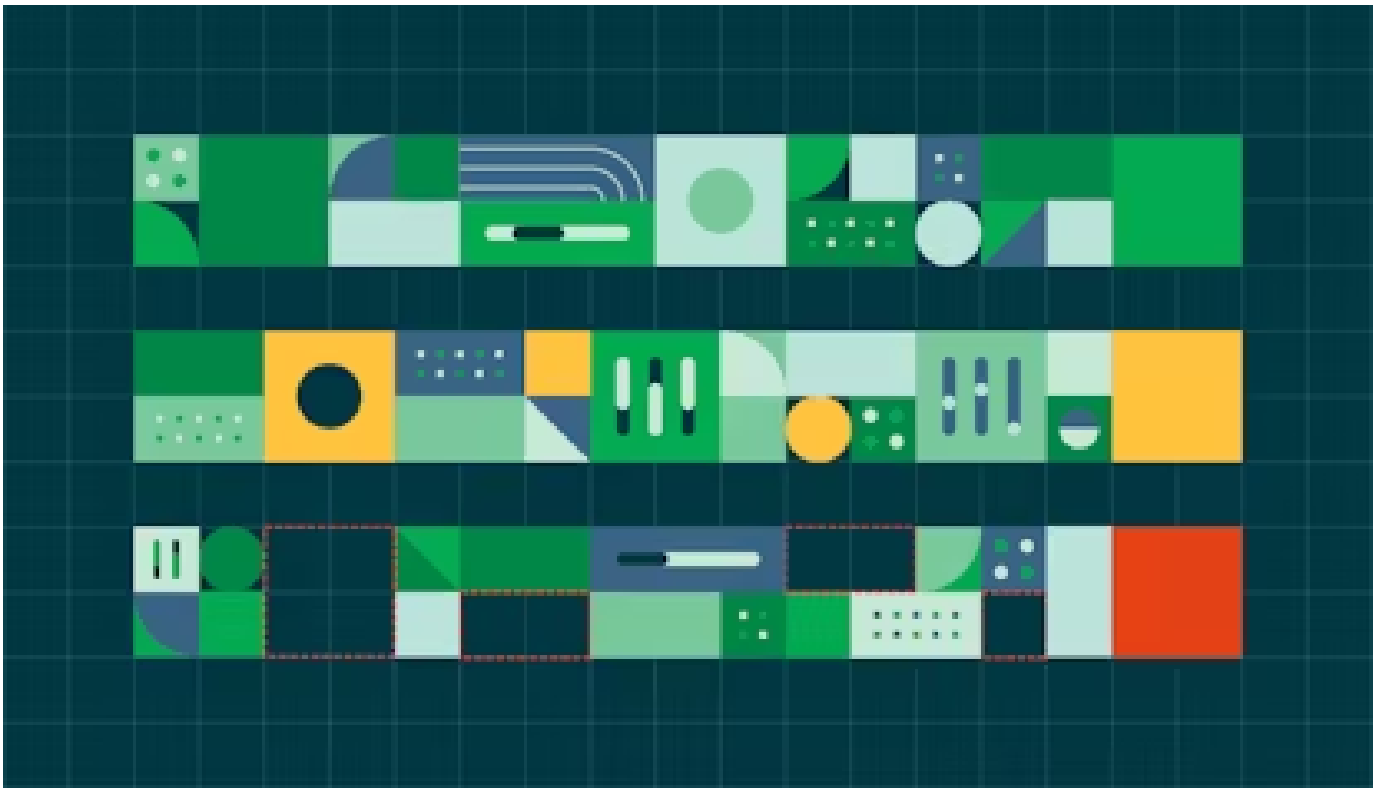
(/blog/android-parallel-testing/)

Tutorials (/blog/tag/tutorials/)

Splitting and parallelizing Android UI tests with Espresso and CircleCI (/blog/android-parallel-testing/)

MAR 7, 2024 | 5 MIN READ

Tadashi Nemoto (/blog/author/tadashi-nemoto/)
Senior Solutions Engineer



(/blog/llm-hallucinations-ci/)

Tutorials (/blog/tag/tutorials/)

LLM hallucinations: How to detect and prevent them with CI (/blog/llm-hallucinations-ci/)

JAN 24, 2024 | 9 MIN READ

Michael Webster (/blog/author/michael-webster/)
Principal Engineer

(/blog/testing-pytorch-model-with-pytest/)

Tutorials (/blog/tag/tutorials/)

Testing a PyTorch machine learning model with pytest and CircleCI (/blog/testing-pytorch-model-with-pytest/)

DEC 8, 2023 | 9 MIN READ

Vivek Maskara (/blog/author/vivek-maskara/)
Software Engineer

circleci (https://circleci.com/)